# Automata-Theoretic Verification of Interval Markov Decision Processes

Sarvin Bahmani[1], Soumyajit Paul[1], Sven Schewe[1], Sadegh Soudjani[2] and Ashutosh Trivedi[1,3]

## I. Abstract

***Abstract*— Interval Markov decision processes (IMDPs) provide a natural framework for modeling stochastic systems with uncertain transition probabilities, represented by probability intervals and resolved adversarially. Such uncertainty arises naturally, for example, when the transition model is learned from finite data or obtained through model-based reinforcement learning. In this paper, we study the automata-theoretic verification of IMDPs against rich temporal specifications, including all LTL specifications, by considering the broader class of $\omega$-regular objectives. We show that classical automata-theoretic verification techniques extend to IMDPs, but with a sharp distinction determined by the structure of the transition intervals. For stable IMDPs, where interval bounds exclude zero, verification reduces to ordinary MDP analysis and can be carried out using the standard automata used in that setting (good-for-MDP automata). For unstable IMDPs, where intervals may include zero, verification becomes game-like and requires automata whose nondeterminism can be resolved online (good-for-games automata). Building on these insights, we develop algorithms for verifying $\omega$-regular specifications over IMDPs and derive probabilistic guarantees when the interval model is learned from sampled data. The resulting framework enables principled verification of stochastic systems under probabilistic model uncertainty, connecting automata-based verification with data-driven stochastic modeling.**

## II. Introduction

Stochastic systems in control, robotics, and learning are often modeled only approximately. In many settings, transition probabilities are not known exactly, but must be estimated from finite data or inferred through model-based reinforcement learning. This uncertainty makes formal reasoning difficult, particularly when one seeks to verify rich temporal properties rather than simple reachability or safety objectives.

A natural framework for capturing such uncertainty is provided by interval Markov decision processes (IMDPs), in which each transition probability is specified by an interval rather than a fixed value. The interval representation compactly describes a family of stochastic models and supports robust reasoning by interpreting the uncertainty adversarially. IMDPs have therefore emerged as a useful abstraction for data-driven stochastic systems, where exact transition laws are unavailable but confidence bounds or structural regularity assumptions allow one to bound transition probabilities.

To specify desired behaviors, the formal methods literature has long relied on temporal logics such as linear temporal logic (LTL) and, more generally, on the expressive class of $\omega$-regular objectives [1], [2], [3]. Automata-theoretic methods provide a powerful bridge between such specifications and algorithmic verification. This raises a natural question: how far do these classical verification techniques extend when the underlying system is an IMDP rather than an MDP?

In this work, we study the automata-theoretic verification and control of IMDPs against $\omega$-regular objectives, including all LTL specifications. Our goal is to understand when interval uncertainty preserves the classical MDP view of automata-based verification and when it instead induces a genuinely game-like setting that requires stronger automata.

**Interval MDPs.** In data-driven stochastic control, IMDPs arise by abstracting an uncertain system into finitely many states and bounding transition probabilities by intervals derived from samples or structural assumptions on the dynamics. This yields a finite model that represents a family of stochastic systems and supports robust verification under adversarially resolved uncertainty.

Constructing an IMDP from data typically involves partitioning the continuous state space into finitely many regions and estimating possible transitions between regions under different control inputs. Rather than relying on exact probability estimates, the abstraction bounds transition probabilities using intervals derived from statistical confidence bounds, regularity assumptions such as Lipschitz continuity, or other structural properties of the dynamics. Once an IMDP is available, one can apply probabilistic verification and policy-synthesis techniques to analyze temporal specifications under model uncertainty.

Recent work has developed methods for constructing IMDP abstractions from data. Badings et al. [4] consider systems with known linear dynamics and additive noise of unknown distribution, yielding IMDPs with a fixed underlying graph structure. We refer to such IMDPs as *stable*. Nazeri et al. [5] study fully unknown nonlinear systems with a known Lipschitz constant, yielding abstractions in which some transition intervals may include zero, so that the underlying graph may depend on how the uncertainty is resolved. We refer to such IMDPs as *unstable*. Both works focus on reach-avoid objectives and compute optimal probabilities on the IMDP abstraction, thereby obtaining guarantees for probabilistic reachability and safety.

**Which automata are good for IMDPs?** Automata-theoretic verification of $\omega$-regular objectives typically proceeds by translating the specification into an automaton and forming a product with the underlying stochastic model. A central difficulty is that expressive automata often involve nondeterminism. For such automata to be useful algorithmically, their nondeterministic choices must be resolvable in a way that preserves the satisfaction probability of the specification. For

[1] University of Liverpool, UK
[2] University of Birmingham, UK, and MPI-SWS, Germany
[3] University of Colorado Boulder, Colorado, USA

ordinary MDPs, this difficulty is mitigated by the fact that the controller interacts only with stochastic dynamics rather than a strategic adversary. In this setting, *good-for-MDP (GFM)* automata [6], [7], [8], [9] suffice: their nondeterminism can be resolved without distorting satisfaction probabilities.

For IMDPs, transition uncertainty is resolved adversarially. Thus, although there is only one explicit controller, the interval uncertainty acts as an adversarial environment. Verification is therefore semantically closer to a game than to a standard MDP. In this setting, resolving automaton nondeterminism from the observed history alone may no longer preserve correctness. This is precisely where the stronger *good-for-games (GFG)* [10] property becomes relevant: it requires that nondeterminism be resolved online, based only on the past, against an adversarial environment.

This observation reveals the central challenge addressed in this paper: interval uncertainty changes the semantics of automata-based verification. Automata that are sound for MDPs need not remain sound for IMDPs, but where they are, it allows for using significantly faster algorithms. The precise boundary depends on whether the uncertainty preserves or can alter the connectivity structure of the model.

**Contributions.** We study verification and control of IMDPs against $\omega$-regular objectives. Our main contributions are:

1) We identify the automata-theoretic boundary for IMDP verification. In particular, we show that GFM automata suffice for stable IMDPs, where interval bounds exclude zero so that the adversarial player cannot alter the connectivity structure, whereas unstable IMDPs require the stronger GFG property.
2) We develop algorithms for computing optimal and robust policies for IMDPs with $\omega$-regular objectives, combining automata-theoretic constructions with insights from probabilistic parity games [11], [12].
3) We implement two solution procedures: (i) a value iteration method that introduces slight imprecision because of the contraction used for upper bounds, and (ii) a strategy-improvement method that is slightly more expensive but exact. Unlike previous approaches, our method iterates over nature strategies, yielding small linear programs instead of exponential-size resolver programs. We evaluate the approach on large-scale case studies with diverse $\omega$-regular objectives, demonstrating scalability and practical applicability.

Proofs and more details can be found in extended version.

## III. Preliminaries

A *probability distribution* over a set $X$ is a function $d\colon X\to[0,1]$ such that $\sum_{x\in X} d(x)=1$. Let $\mathbf{Dis}(X)$ denote the set of all distributions over $X$. For a finite or infinite sequence $s=\langle x_1,x_2,\ldots\rangle$, we write $s(k)$ to denote its $k$-th element. For a finite sequence $s=\langle x_1,\ldots,x_n\rangle$ and an element $y$, we write $s\cdot y$ for the sequence $\langle x_1,\ldots,x_n,y\rangle$. For a finite sequence $s=\langle x_1,x_2,\ldots,x_n\rangle$, we write $\mathsf{last}(s)$ for the final element $x_n$ of $s$.

*Definition 1 (IMDP):* An Interval MDP (IMDP) $\mathcal{M}$ is a tuple $(X,x_0,A,\overline{P},\underline{P},\mathcal{AP},L)$ where $X$ is a finite set of states; $x_0\in X$ as the initial state; $A$ is a finite set of actions; $\overline{P}: X\times A\times X\to[0,1]$ and $\underline{P}: X\times A\times X\to[0,1]$ are upper and lower transition probability bound functions; $\mathcal{AP}$ is a finite set of atomic propositions and $L: X\to 2^{\mathcal{AP}}$ is a labeling function. Furthermore, to ensure $\overline{P}$ and $\underline{P}$ admit well-formed transition probabilities, we require that, for any $x,x'\in X$ and $a\in A$, we have $\underline{P}(x,x',a)\le\overline{P}(x,x',a)$, and

$$\sum_{x'\in X}\underline{P}(x,x',a)\le 1\le\sum_{x'\in X}\overline{P}(x,x',a).$$

When $\overline{P}=\underline{P}$, the IMDP becomes an MDP.
Given a state $x\in X$ and input $a\in A$, we denote by $\Delta(x,a)$ the set of feasible distribution over $X$, i.e.,

$$\Delta(x,a)=\Big\{p\in\mathbf{Dis}(X):\sum_{x'\in X}p(x'|x,a)=1 \text{ and } \underline{P}(x,x',a)\le p(x'|x,a)\le\overline{P}(x,x',a) \text{ for all } x'\in X\Big\}. \quad (1)$$

Note that $\Delta(x,a)$ is a polytope in $\mathbb{R}^{|X|}$. Given a state $x\in X$, we denote the set of all actions that are enabled at state $x$ by $\Gamma(x)=\{a\in A\mid \text{there exists } x'\in X, \text{ such that } \overline{P}(x,x',a)>0\}$. A finite path of an IMDP $\mathcal{M}=(X,x_0,A,\overline{P},\underline{P},\mathcal{AP},L)$ is a finite sequence of states $s=\langle x_1,x_2,\ldots,x_n\rangle$ where, $x_1=x_0$, for all $1<i\le n$, there exists $a\in\Gamma(x_{i-1})$ such that $\overline{P}(x_{i-1},x_i,a)>0$. An infinite path is an infinite such sequence. We denote by $\mathsf{Path}^*(\mathcal{M})$ and $\mathsf{Path}^\omega(\mathcal{M})$ the set of all finite paths and infinite paths in $\mathcal{M}$, respectively. Let $\mathsf{Path}(\mathcal{M})=\mathsf{Path}^*(\mathcal{M})\cup\mathsf{Path}^\omega(\mathcal{M})$ be the set of all paths.

**The policy and the nature.** A *policy* is a function $\pi:\mathsf{Path}^*(\mathcal{M})\to\mathbf{Dis}(A)$, which assigns the distribution over action set to each finite path. A *nature* is a function $\theta:\mathsf{Path}^*(\mathcal{M})\times A\to\mathbf{Dis}(X)$, such that, for every finite path $s\in\mathsf{Path}^*(\mathcal{M})$ and action $a\in A$, the distribution $\theta(s,a)$ is a feasible successor distribution in $\Delta(\mathsf{last}(s),a)$. Let $\Pi(\mathcal{M})$ and $\Theta(\mathcal{M})$ be the sets of all policies and natures for $\mathcal{M}$.

For $(\pi,\theta)\in\Pi(\mathcal{M})\times\Theta(\mathcal{M})$, let $\mathsf{Path}_{\mathcal{M}}^{\pi,\theta}$ denote the set of infinite runs of $\mathcal{M}$ starting at $x_0$ that are consistent with $(\pi,\theta)$. Given a finite path $s=\langle x_1,x_2,\ldots,x_n\rangle\in\mathsf{Path}^*(\mathcal{M})$, the *cylinder set* generated by $s$ is

$$\mathsf{Cyl}(s)=\big\{\rho\in\mathsf{Path}^\omega(\mathcal{M})\,\big|\,\rho_i=s_i \text{ for } 1\le i\le n\big\}.$$

Let $\mathcal{F}_{\mathcal{M}}^{\pi,\theta}$ be the $\sigma$-algebra generated by cylinder sets. Then the behavior of $\mathcal{M}$ under $(\pi,\theta)$ is described by the probability space $(\mathsf{Path}_{\mathcal{M}}^{\pi,\theta},\mathcal{F}_{\mathcal{M}}^{\pi,\theta},\Pr_{\mathcal{M}}^{\pi,\theta})$, where the measure $\Pr_{\mathcal{M}}^{\pi,\theta}$ is uniquely determined by the Ionescu–Tulcea extension theorem. Given some $f:\mathsf{Path}^\omega(\mathcal{M})\to\mathbb{R}$, we write $\mathbb{E}_{\mathcal{M}}^{\pi,\theta}[f]$ for its expectation with respect to this probability space.

**Parity IMDPs.** When analyzing IMDP with parity objectives, we consider *parity IMDP*, which are tuples $\mathcal{M}=(X,x_0,A,\overline{P},\underline{P},\alpha)$, where $(X,x_o,A,\overline{P},\underline{P})$ is an IMDP and $\alpha:X\to\mathbb{N}$ is a priority function that maps states to natural numbers, called *colors*. For infinite paths in the IMDP, we call the highest color that occurs infinitely often *dominating*, and the path *accepting* if the dominating color is even. For

a parity IMDP $\mathcal{M}$, a policy $\pi$ and a nature $\theta$, we let

$$\mathsf{Pr}_{\mathcal{M}}^{\pi,\theta} = \Pr_{\mathcal{M}}^{\pi,\theta}\left\{ s \in \mathsf{Path}_{\mathcal{M}}^{\pi,\theta} \ : \ s \text{ is an accepting path} \right\}.$$

Let $\mathsf{Pr}_{\mathcal{M}}^{\pi} = \inf_\theta \mathsf{Pr}_{\mathcal{M}}^{\pi,\theta}$ be the (guaranteed) probability that $\mathcal{M}$ creates an accepting path (regardless of nature) under policy $\pi$, and $\mathsf{Pr}_{\mathcal{M}} = \sup_\pi \mathsf{Pr}_{\mathcal{M}}^{\pi}$ be the (achievable) probability that $\mathcal{M}$ creates an accepting path for any policy $\pi$.

**Stochastic parity games.** A stochastic parity game (SPG) $\mathcal{G}$ is a tuple $(X_e, X_o, X_r, x_0, P, E, \alpha)$ where $X = X_e \cup X_o \cup X_r$ is a finite set of states, partitioned into states $X_e$ owned by Player maximizer, states $X_o$ owned by Player minimizer, and random states $X_r$; $x_0 \in X$ is the initial state; $\alpha : X \to \mathbb{N}$ a coloring function; $P : X_r \times X \to [0,1]$ is a probability function, so that $\sum_{x \in X} P(x', x) = 1$ holds for all $x' \in X_r$; and $E \subseteq (X_e \cup X_o) \times X$ is a transition relation that maps at least one successor to each state in $X_e \cup X_o$. SPGs are played by placing a token on $x_0$. When the token is on a state in $X_e$, the maximizer chooses the successor state from $E$, when it is on a state in $X_o$, the minimizer chooses, and for states in $X_r$, the successor state is sampled with the probabilities from $P$. The objective of the maximizer (minimizer) is to maximize (minimize) the chance that the dominating color in the ensuing play is even.

**Parity and Büchi Automata.** A deterministic parity automaton $\mathcal{D}$ is a tuple $\mathcal{D} = (\mathcal{AP}, Q, q_0, \delta, \alpha)$, where $\mathcal{AP}$ is a set of atomic propositions, $Q$ is a finite set of states, $q_0 \in Q$ is the initial state, $\delta \colon Q \times 2^{\mathcal{AP}} \to Q$ is the transition function, and $\alpha \colon Q \to \mathbb{N}$ is the coloring function. A deterministic parity automaton reads an infinite word over the alphabet $2^{\mathcal{AP}}$ and induces a unique infinite run over $Q$ starting from $q_0$. A run is accepting if the highest color that appears infinitely often along the run is even. The set of words whose runs are accepting is the language of the automaton.

A nondeterministic parity automaton $\mathcal{N}$ is a tuple $\mathcal{N} = (\mathcal{AP}, Q, Q_0, \Delta, \alpha)$ defined as above, except that the set of initial states $Q_0$ need not be a singleton, and the transition function $\Delta : Q \times 2^{\mathcal{AP}} \to 2^Q$ maps each state and input letter to a set of successor states. As a result, a nondeterministic automaton typically admits multiple runs on a given word, and its language consists of all words for which at least one run is accepting. Nondeterministic automata $\mathcal{N}$ with the property that resolving the nondeterminism in $\mathcal{N}$ yields, for every finite MDP $\mathcal{M}$, the same value as computing an accepting run in the parity MDP obtained from the product of $\mathcal{M}$ and $\mathcal{N}$ are called *good-for-MDPs*. Parity conditions for which the image of $\alpha$ is $1, 2$ are called *Büchi* conditions.

## IV. What's Good for IMDPs?

We develop automata-theoretic algorithms for analyzing IMDPs, which raises the question of which automata are appropriate for this setting. We show that this question turns crucially on whether transition intervals include zero, since zero-valued lower bounds allow the environment to suppress certain transitions altogether. This feature typically depends on the construction of the IMDP: some abstraction methods yield such intervals, whereas others avoid them.

*Definition 2 (Stable IMDPs):* We call an IMDP *stable* if, for all $x, x' \in X$ and $a \in A$, either $\underline{P}(x, x', a) > 0$ or $\overline{P}(x, x', a) = 0$ holds. Intuitively, stability ensures that every transition allowed by the abstraction remains possible under all admissible probability selections of the environment.

In this paper, we focus primarily on stable IMDPs (SMDPs). For this class, we show that qualitative analysis coincides with that of ordinary MDPs, and therefore good-for-MDP Büchi automata [6] suffice to capture $\omega$-regular objectives. Moreover, quantitative analysis reduces to reachability, allowing the use of standard techniques for reachability games, such as value iteration and strategy improvement. This reduction is specific to stable IMDPs and does not hold for general IMDPs.

Before turning to SMDPs, we first outline an approach for analyzing general IMDPs. The main idea is to construct the synchronous product of an IMDP with a suitable automaton whose memory acts as a witness for acceptance. For general IMDPs, this automaton must be a deterministic parity automaton, or more generally a GFG parity automaton [13], recognizing the target $\omega$-regular language on $2^{\mathcal{AP}}$.

To analyze an IMDP $\mathcal{M} = (X, x_0, A, \overline{P}, \underline{P}, \mathcal{AP}, L)$ against a specification given by a deterministic parity automaton $\mathcal{D} = (\Sigma, Q, q_0, \delta, \alpha)$, we construct a stochastic parity game $\mathcal{G}$ as follows:

- **Maximizer states:** The states are $X \times Q$, with initial state $(x_0, q_0)$.
- **Maximizer choices:** From a state $(x, q)$, the maximizer chooses an action $a \in \Gamma(x)$ and moves to the corresponding minimizer state $(x, q, a)$.
- **Minimizer states:** These are states $(x, q, a) \in X \times Q \times A$ with $a \in \Gamma(x)$. From such a state, the minimizer selects a corner $p$ of the polytope $\Delta(x, a)$, yielding a random state $(x, q, a, p)$.[1]
- **Random states:** From $(x, q, a, p)$, the game moves to a successor $(x', q')$ with probability $p(x' \mid x, a)$, where $q' = \delta(q, L(x))$.
- **Colors:** The coloring $\alpha'$ is inherited from $\mathcal{D}$:

$$\alpha'(x, q) = \alpha(q), \ \alpha'(x, q, a) = \alpha(q), \ \alpha'(x, q, a, p) = \alpha(q).$$

Alternatively, we may construct a parity IMDP $\mathcal{P} = \big(X \times Q, (x_0, q_0), A, \overline{P}', \underline{P}', \alpha'\big)$, in which the automaton component is updated deterministically according to $q' = \delta(q, L(x))$, while the system component evolves as in $\mathcal{M}$. Hence, the transition bounds $\overline{P}'$ and $\underline{P}'$ depend only on the $X$-component, and the coloring is given by $\alpha'(x, q) = \alpha(q)$.

Note that $\mathcal{G}$ may be exponentially larger than $\mathcal{M}$. This is because $\Delta(x, a)$ may have exponentially many vertices, each inducing a random state, even when $\mathcal{M}$ is an SMDP. For example, if the support of $\Delta(x, a)$ has size $2k + 1$ for some $k \in \mathbb{N}$ and each interval is $\left[\frac{1}{2k+2}, \frac{3}{2k+2}\right]$, then $\Delta(x, a)$ has $(2k + 1)\binom{2k}{k}$ states. Each vertex corresponds to a distribution in which one successor has probability

[1] A corner is a point that cannot be written as a nontrivial convex combination of two distinct points of the polytope. Any probabilistic choice in $\mathcal{P}$ can be expressed as an affine combination of these vertices.

$\frac{1}{k+1}$, exactly $k$ successors have probability $\frac{1}{2k+2}$, and the remaining $k$ have probability $\frac{3}{2k+2}$.

*Lemma 4.1:* For every policy $\pi$ and every nature $\theta$, the measure $\Pr_{\mathcal{P}}^{\pi,\theta}$ coincides with the probability that $\mathcal{M}$ generates a path whose label trace is accepted by $\mathcal{D}$.

*Proof:* By construction of the product, $\mathcal{P}$ and $\mathcal{M}$ induce the same probability space over paths, generated by the same choices of $\pi$ and $\theta$. Moreover, along each path, the automaton component is updated exactly according to $\delta$, so the color sequence in $\mathcal{P}$ matches the acceptance condition of $\mathcal{D}$ on the corresponding label trace of $\mathcal{M}$. Hence, a path is accepting in $\mathcal{P}$ if and only if its trace in $\mathcal{M}$ is accepted by $\mathcal{D}$. ∎

*Lemma 4.2:* The maximizer has the same pure positional optimal policies, achieving the same winning probabilities, in $\mathcal{P}$ and in $\mathcal{G}$.

*Proof:* Finite SPGs admit optimal pure positional strategies [14], [15]. Let $p$ denote the probability of satisfying the parity objective in $\mathcal{G}$ when both players follow optimal pure positional policies $\pi : X \times Q \to A$ and $\theta : X \times Q \times A \to (X \times Q \to [0,1])$. Note that each transition in $\mathcal{P}$ is expanded into three steps in $\mathcal{G}$, but this affects only the encoding of paths, not the induced values or optimal choices.

We now show that these policies are also optimal in $\mathcal{P}$ and yield the same probability $p$.

1) Suppose for contradiction that there exists a policy $\pi'$ in $\mathcal{P}$ that achieves a higher probability $p' > p$ against nature $\theta$. Then $\pi'$ induces a corresponding policy against $\theta$ in $\mathcal{G}$, yielding the same winning probability $p' > p$, contradicting the optimality of $\pi$.
2) Conversely, suppose for contradiction that there exists a nature strategy $\theta'$ in $\mathcal{P}$ that achieves a lower winning probability $p' < p$ against $\pi$. Then $\theta'$ induces a corresponding strategy against $\pi$ in $\mathcal{G}$, again yielding probability $p' < p$, contradicting the optimality of $\theta$.

Therefore, $\pi$ and $\theta$ remain optimal in $\mathcal{P}$, and the winning probability is the same in $\mathcal{P}$ and $\mathcal{G}$. ∎

Efficient algorithms for both qualitative [16] and quantitative [17] analysis of stochastic parity games are available. In particular, the qualitative analysis can, in principle, be solved via a gadget-based reduction to non-stochastic parity games [18], which can then be handled in quasi-polynomial time [19], [20], [21]. For an IMDP $\mathcal{M}$, the *qualitative analysis* problem is determining the set of states $x \in X$ for which $\Pr_{\mathcal{M}_x} = 0$ or $\Pr_{\mathcal{M}_x} = 1$ holds (where $\mathcal{M}_x = (X, x, A, \overline{P}, \underline{P}, \alpha)$).

*Theorem 4.3:* The qualitative analysis of IMDPs is EXPTIME-complete when the property is given as a nondeterministic Büchi automaton, and 2EXPTIME-complete when the property is given as an LTL formula.

*Proof:* For the upper bound, note that an NBA $\mathcal{N}$ with $s$ states can be determinized into a parity automaton $\mathcal{D}$ with an exponential number of states ($s!^{2}$) and with linearly many priorities ($2s+1$). The resulting stochastic parity game $\mathcal{G}$ has the same priorities as $\mathcal{D}$ and a number of states that is exponential in both $\mathcal{D}$ and $\mathcal{M}$. Qualitative analysis of $\mathcal{G}$ can then be performed in time exponential in the sizes of $\mathcal{M}$ and $\mathcal{N}$ (doubly exponential in the size of an LTL formula $\varphi$). Matching lower bounds follow from the complexity of MDP analysis [7]. ∎

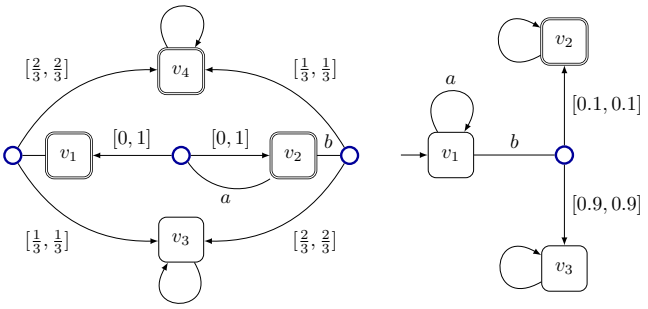



Fig. 1: (Left) Büchi IMDP where the maximizer can win from $v_2$ with probability $\frac{2}{3}$ by always playing $a$, but reaches the almost-sure winning region only with probability $\frac{1}{3}$. (Right) Reachability IMDP, where interval value iteration does not terminate without contraction of the upper value. Upper value remains strictly greater than lower value by a fixed margin after few steps.

### A. Quantitative Analysis

We now establish three upper bounds for the *quantitative analysis* problem, i.e., computing the maximal satisfaction probability of an IMDP $\mathcal{M}$ against an NBA $\mathcal{N}$.

*Theorem 4.4:* The quantitative analysis of IMDPs against NBA specifications satisfies the following:

1) it is in $\mathsf{UEXPTIME} \cap \mathsf{coUEXPTIME}$, and
2) it can be solved in co-nondeterministic time which is polynomial in the size of the IMDP and exponential in the size of the NBA.

*Proof:* For (1), we use that stochastic parity games belong to a class of stochastic games that are polynomial-time interreducible [22] and that membership of parity games in $\mathsf{UP} \cap \mathsf{coUP}$ is well established (see, e.g., [23] for discounted-payoff games, which admit a unique value). For (2), we exploit that both players have optimal pure positional strategies. To refute an inequality of the form $\Pr_{\mathcal{M}} \geq p_0$, it suffices to nondeterministically guess a pure positional minimizer strategy. This guess can be made incrementally while constructing the game, guaranteeing that each minimizer state has a single successor. The resulting structure is then an ordinary parity MDP of size polynomial in $\mathcal{M}$ and exponential in $\mathcal{N}$ (via the determinization of $\mathcal{N}$ into a parity automaton $\mathcal{D}$). Solving such parity MDPs can be done in polynomial time. ∎

As these bounds are expressed in terms of $\mathcal{N}$, the corresponding upper bounds for LTL follow directly from translating an LTL formula into an equivalent, but potentially exponentially larger, NBA $\mathcal{N}$.

We note that, while we can in principle reduce the problem to a reachability game using the construction from [22], this reduction introduces gadgets with extremely small probabilities. In particular, these are *not* reductions to the reachability

of the almost sure (a.s.) winning region from the qualitative analysis, and such reductions would not yield correct results. This is shown in Fig. 1(left), which also shows why naive value iteration fails for these games.

*Example 1: (Reachability of a.s. Winning Regions vs. Winning)*: In Fig. 1(left), the a.s. winning region is $\{v_4\}$. If the maximizer wants to maximize her chances of reaching it from $v_2$, her best move is to choose action $b$, because the minimizer can push her back to state $v_2$ if she plays $a$. Playing $b$ reaches $v_2$ and wins with a chance of $\frac{1}{3}$, while it reaches reaching $v_3$ and loses with a chance of $\frac{2}{3}$. The best strategy for the maximizer, however, is to always play $a$ from state $v_2$. While this does not guarantee reach her winning (almost) sure winning as the minimizer can always play back to $v_2$, the minimizer loses if he does this for ever, and always playing $a$ guarantees a chance of winning of at least $\frac{2}{3}$ to the maximizer. This shows that maximizing the chance of reaching an a.s. winning region is not the same as maximizing the chance of winning.

### B. Good-for-IMDP automata

We have presented the product construction using deterministic automata only for simplicity of exposition. The same construction applies equally well to good-for-games automata [13], [24] that recognize the same language. Conversely, labeled stochastic games can be encoded as IMDPs in a straightforward manner. It follows that the class of automata suitable for IMDP analysis cannot be more general than the class of automata suitable for games.

## V. Value Iteration and Strategy Improvement

We propose two algorithms for the verification of stable IMDPs against $\omega$-regular properties. We have implemented these algorithms and report experimental results in Section VI. The first algorithm is a value iteration algorithm, while the second one is a variant of strategy improvement. The (interval) value iteration algorithm keeps track of upper and lower values, and deploys a contraction factor for the upper value. The strategy improvement differs from existing strategy improvement algorithms for IMDP, and iterates over nature instead of policies. The first step in both the algorithms is a qualitative analysis of the SMDP. This procedure is identical to MDP qualitative analysis.

*a) Qualitative analysis:* The qualitative analysis is performed by ignoring the exact probability values, since in SMDP all probabilities are positive. Recall that this is essentially computing the almost-sure winning regions and sure losing regions of some reachability game, that we describe shortly. Given an SMDP $\mathcal{M}$ and an automata $\mathcal{A} = (\Sigma, Q, q_0, \Delta, \alpha)$, we compute the product $\mathcal{M} \times \mathcal{A}$, with states $X \times Q$. On the underlying MDP structure comprising of all positive probabilities in $\mathcal{M}$, we first identify the accepting maximal end components of $\mathcal{M} \times \mathcal{A}$ and then collapse them into a target (winning sink) $T$, thus forming a reachability game. We compute the almost sure winning region $W^+$, from where $T$ can be reached almost surely. The sure losing region $W^-$ are those states, from which the targets cannot be reached under any policy and collapse them into a losing sink. These computations determine initial values for our value iteration step.

*Theorem 5.1:* Given an SMDP $\mathcal{M}$ and automaton $\mathcal{N}$, the qualitative analysis of $\mathcal{M}$ can be done in time polynomial in $\mathcal{M}$ and exponential in $\mathcal{N}$ if $\mathcal{N}$ is an NBA. If $\mathcal{N}$ is good for MDPs, this can be done in time polynomial in $\mathcal{M}$ and $\mathcal{N}$. The corresponding matching lower bounds are inherited from the MDP case [7].

In the second step we perform quantitative analysis of SMDPs. Similar to MDPs, this reduces to a reachability game but with an added adversarial nature.

### A. Interval Value Iteration

Our first quantitative procedure is an interval value iteration algorithm (Algorithm 1), in the spirit of the interval iteration method of [25]. In the first step, we identify two regions: $W^+$, the almost-sure winning region, and $W^-$, the sure losing region. The algorithm then initializes, for each state, a lower and an upper value. For states in $W^+$, both values are set to 1; for states in $W^-$, both are set to 0; and for all other states, the lower and upper values are initialized to 0 and 1, respectively.

In each iteration, the values of states in $W^+$ and $W^-$ remain fixed. For every other state, the values are updated as follows. First, for each $(s, a)$, nature chooses transition probabilities $\theta(s, a, s')$ adversarially within the prescribed intervals using the local best-response algorithm of [26], applied separately to the current lower and upper values. This minimizes $\sum_{s' \in S} \theta(s, a, s')V(s')$, where $V$ is instantiated by the current lower or upper value function (Algorithm 2). The lower and upper values are then updated in the usual way by taking the weighted average of successor values. For the upper bound, we additionally multiply by a contraction factor $\alpha < 1$.

The algorithm terminates once the upper value is no greater than the lower value at every state. Standard arguments show that Algorithm 1 terminates after finitely many iterations. The contraction factor is essential here: without contraction, the upper value at $v_1$ remains 1 in every iteration, while the lower value quickly drops to $0.1$, so that the gap eventually stabilizes. With contraction, however, the upper value eventually falls below $0.1$.

The algorithm also provides a basic approximation guarantee: by choosing $\alpha$ sufficiently close to 1, the final value can be made arbitrarily close to the true value.

*Theorem 5.2:* For an IMDP $\mathcal{M}$, Algorithm 1 always terminates with a value $v \leq \mathsf{Pr}_{\mathcal{M}}$. Moreover, for every $\varepsilon > 0$, there exists $\alpha < 1$ such that $v \geq (1 - \varepsilon)\mathsf{Pr}_{\mathcal{M}}$.

### B. Nature Strategy Improvement Algorithm

In our Algorithm 3, we use strategy improvement for exact quantitative analysis using. After solving the almost sure winning and sure losing regions, we use strategy improvement, where each iteration (lines 4-7) consists of finding a new nature (lines 5,6) using the same subroutine Algorithm 2

**Algorithm 1:** Value Iteration for IMDPs with A.S. and Losing Regions

**Input:** Regions $W^+$, $W^-$; discount factor $\alpha < 1$
**Output:** Approximate lower and upper value functions $V^-, V^+$

**foreach** $s$ **do**
  $(V^-(s), V^+(s)) \leftarrow \begin{cases} (1,1) & \text{if } s \in W^+, \\ (0,0) & \text{if } s \in W^-, \\ (0,1) & \text{otherwise.} \end{cases}$
**while** $\exists s$ *such that* $V^-(s) < V^+(s)$ **do**
  **foreach** $s \notin W^+ \cup W^-$ **do**
    **foreach** *action* $a$ *enabled at* $s$ **do**
      compute $(\theta^-_{s,a,s'}, \theta^+_{s,a,s'})$ via Algo. 2;
    $V^-(s) \leftarrow \max_a \sum_{s'} \theta^-_{s,a,s'} V^-(s')$;
    $V^+(s) \leftarrow \max_a \alpha \sum_{s'} \theta^+_{s,a,s'} V^+(s')$;
**return** $V^-, V^+$;

**Algorithm 2:** Best Local Nature Subroutine

**Input:** Values $V(S)_{s \in S}$
**Output:** Best local nature $\theta$ minimising $\sum_{s' \in S} \theta(s,a,s') V(s')$ for each $s, a$
**foreach** $(s,a,s')$ **do**
  $\theta(s,a,s') \leftarrow \underline{P}(s,a,s')$;
Sort states w.r.t. $V(s)$ into $s'_0, \ldots, s'_{n-1}$ ;
**foreach** $(s,a)$ **do**
  $i \leftarrow 0$;
  **while** $\sum_{s' \in S} \theta(s,a,s') V(s')$ < *1* **do**
    $\theta(s,a,s'_i) \leftarrow \min(\overline{P}(s,a,s'_i), 1 - \sum_{s' \in S} \theta(s,a,s') V(s')))$;
    $i \leftarrow i + 1$;
**return** $\theta$;

used for value iteration, followed by finding an optimal policy for this nature (line 7).

Existing strategy improvement [26] for IMDP do this reversely: they iterate over policies and then look for the best nature response for these policies in every iteration. While this difference seems minor, computing the globally best response of nature is expensive: it entails choosing an optimal corner for nature among exponentially many vertices in the polytope formed by the $\underline{P}$ and $\overline{P}$ values, and this create a linear program of exponential size (one variable for every corner of each polytope), whereas the linear programme we generate in line 7 incurs no blow-up.

*Theorem 5.3:* Algorithm 3 terminates with the optimal strategy and values.

In an ideal setting with exact rational arithmetic, the natural termination condition would be that the current policy remains optimal. Equivalently, one could stop when the value function does not change, for example when $\sum_{s \in S} V'(s) = \sum_{s \in S} V(s)$. (This is slightly more expensive

**Algorithm 3:** Nature Strategy Improvement

**Input:** $W^+, W^-$
**Output:** $\pi^*, V$

1 Choose arbitrary $\theta'$;
2 Compute best response $(\pi', V')$ to $\theta'$ via LP;
3 **repeat**
4  $\theta, \pi, V \leftarrow \theta', \pi', V'$;
5  **foreach** $(s,a)$ **do**
6  set $\theta'(s,a)$ to the best local nature for $V$;
7  Compute best response $(\pi', V')$ to $\theta'$ via LP;
8 **until** $\sum_{s \in S} V'(s) \geq \sum_{s \in S} V(s)$;
9 **return** $\pi, V$;

as it may require solving the final linear program twice.) However, in practical floating-point implementations, numerical precision can be insufficient to reliably determine whether nature should assign more probability mass to a successor $s$ or to $s'$ when their current values are extremely close. Such marginal differences are purely due to numerical effects. They can lead to the solutions going in cycles with numerically indistinguishable values from the linear programs (e.g. one variable valuation alternating between 2 and 1.999999999999999999999) that mask convergence, as it can lead to $\sum_{s \in S} V'(s) > \sum_{s \in S} V(s)$; we therefore use $\sum_{s \in S} V'(s) \geq \sum_{s \in S} V(s)$ as our stopping criterion.

## VI. Experimental Evaluation

We implemented Algorithms 1 and 3; our implementation is available at `https://github.com/Sarv-Bahmani/IMDPomega`. We evaluated the methods on two benchmarks from DynAbs [27], which constructs IMDP abstractions of stochastic dynamical systems. We consider two case studies: (i) *UAV motion control*, where a drone must be stabilized and guided safely to a target under non-Gaussian disturbances, and (ii) *Shuttle control*, where a spacecraft must track a desired trajectory under uncertain noise. For the UAV benchmark, we varied the partition granularity to study scalability, obtaining models with 787 to 2,430 states and $0.52 \times 10^6$ to $14.9 \times 10^6$ transitions. The Shuttle benchmark contains 3,200 to 7,200 states and $1 \times 10^6$ to $6 \times 10^6$ transitions. Each IMDP was checked for stability to ensure that our quantitative analysis applies.

For both case studies, we considered a collection of $\omega$-regular specifications expressed in LTL and translated them into good-for-MDP Büchi automata using Spot [28]. Figure 3 reports the runtime of SI, VI, and qualitative analysis for the UAV and Shuttle benchmarks as a function of the number of transitions, with each curve corresponding to a specification. Overall, the results show that our framework scales to large IMDPs while supporting a broad range of temporal specifications. As expected, runtime increases with model size for all three procedures. Qualitative analysis remains relatively inexpensive, whereas quantitative analysis dominates the overall cost. SI is sometimes slower in prac-

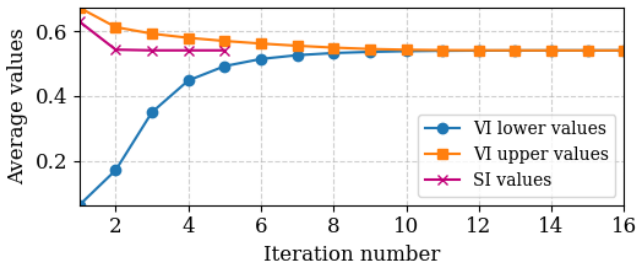
tice because of its higher per-iteration cost, but its nature-improvement scheme makes its runtime comparable to VI.

The experiments also highlight the influence of the specification on computational cost. Simple reachability formulas, such as $\mathbf{F}\,reach$, are handled efficiently, while formulas with nested temporal operators or combined safety and liveness requirements lead to larger automata and higher runtimes. In particular, specifications such as $\mathbf{FG}(reach) \wedge \mathbf{G}\neg(deadlock)$ are consistently among the most expensive.

Supporting nondeterministic Büchi automata is important in practice, since determinization may incur exponential blow-up for many LTL formulas. By avoiding determinization, our framework remains expressive while retaining scalability, and can therefore handle richer specifications that would otherwise be impractical.

All experiments were run on the Barkla HPC cluster using AMD EPYC 9334 processors, with 8 CPUs and 64 GB RAM per job under Python 3.12.

Fig. 2: Average initial-state values for VI lower bounds, VI upper bounds, and SI across iterations for an IMDP with 1600 states under $\mathbf{GF}(reach)$.




### A. VI lower/upper bound values vs SI upper bound

To compare VI and SI more directly, we examine their convergence behavior. SI typically converges in fewer iterations, although each iteration is more expensive, leading to similar overall runtimes. Figure 11 shows, for an IMDP with 1,600 states under the specification $\mathbf{GF}(reach)$, the average over the initial states of the VI lower bound, the VI upper bound, and the SI value at each iteration. The VI lower bound increases monotonically, while the VI upper bound decreases until the two meet, at which point Algorithm 1 terminates. The SI curve stabilizes rapidly, reaching its fixed point after about five iterations. The limit of the VI lower and upper bounds matches the SI solution. Thus, VI approaches the same value from below and above, whereas SI computes the exact value for each state.

### B. GR(1) Specifications

Besides full LTL, we also evaluate our framework on the GR(1) fragment. We consider the following GR(1) specifications over the atomic propositions *init*, *reach*, and *deadlock*:

$$\Phi_1: \quad \mathbf{GF}(init) \wedge \mathbf{GF}(\neg deadlock) \rightarrow \mathbf{GF}(reach),$$
$$\Phi_2: \quad \mathbf{GF}(\neg deadlock) \rightarrow \mathbf{GF}(reach) \wedge \mathbf{GF}(init),$$
$$\Phi_3: \quad \mathbf{GF}(\neg deadlock) \rightarrow \mathbf{GF}(reach \wedge \neg init),$$
$$\Phi_4: \quad \mathbf{GF}(init \wedge \neg deadlock) \rightarrow \mathbf{GF}(reach \wedge \neg deadlock).$$

Here, $\Phi_1$ requires recurring success under recurring initialization and infinitely many non-deadlock states. $\Phi_2$ requires repeated recovery and success, namely infinitely many visits to both *init* and *reach*. $\Phi_3$ strengthens this by requiring recurring success outside the initial region. $\Phi_4$ is deadlock-sensitive: whenever the system can repeatedly return to a live initial condition, it must also repeatedly achieve a live success condition. Figure 17 reports the runtime of SI, VI, and qualitative analysis on the UAV benchmarks. The results show that our framework handles GR(1)-induced nondeterminism in large IMDPs and supports richer reactive mission specifications.

## VII. Conclusion

We introduced a framework for verifying and synthesizing $\omega$-regular objectives over interval MDPs, motivated by the broader goal of supporting model-based reinforcement learning. Our analysis focused on *stable* IMDPs, namely those whose concretizations share the same support, and showed that they admit a substantially simpler theory: qualitative analysis coincides with that of ordinary MDPs, while quantitative analysis reduces to reachability.

The significance of stability is not merely theoretical. IMDPs derived from model-based RL naturally satisfy this property, since every observed transition appears with positive probability in the learned dynamics. As a result, the abstraction preserves the connectivity structure needed for automata-theoretic verification. By making the relationship among IMDPs, MDPs, and stochastic games explicit, we also clarified when good-for-MDP automata suffice and when the stronger good-for-games condition becomes necessary for robust satisfaction.

On the algorithmic side, we developed and compared value-iteration- and strategy-improvement-based synthesis procedures. Our empirical evaluation on representative benchmarks shows that both approaches scale well and support end-to-end integration of formal specifications with uncertainty-aware decision making for stable IMDPs with millions of transitions with LTL objectives.

## References


[1] C. Baier and J.-P. Katoen, *Principles of Model Checking*. MIT Press, 2008.

[2] L. de Alfaro, "Formal verification of probabilistic systems," Ph.D. dissertation, 1998.

[3] T. Babiak, F. Blahoudek, A. Duret-Lutz, J. Klein, J. Křetínský, D. Müller, D. Parker, and J. Strejček, "The Hanoi omega-automata format," in *Proc. of CAV*, 2015, pp. 479–486.

[4] T. Badings, L. Romao, A. Abate, D. Parker, H. A. Poonawala, M. Stoelinga, and N. Jansen, "Robust control for dynamical systems with non-gaussian noise via formal abstractions," *Journal of Artificial Intelligence Research*, pp. 341–391, 2023.

[5] M. Nazeri, T. Badings, A.-K. Schmuck, S. Soudjani, and A. Abate, "Data-driven abstraction and synthesis for stochastic systems with unknown dynamics," *arXiv preprint arXiv:2508.15543*, 2025.

[6] E. M. Hahn, M. Perez, S. Schewe, F. Somenzi, A. Trivedi, and D. Wojtczak, "Good-for-MDPs automata for probabilistic analysis and reinforcement learning," in *Proc. of TACAS*, 2020, pp. 306–323.

[7] C. Courcoubetis and M. Yannakakis, "The complexity of probabilistic verification," *J. ACM*, pp. 857–907, 1995.

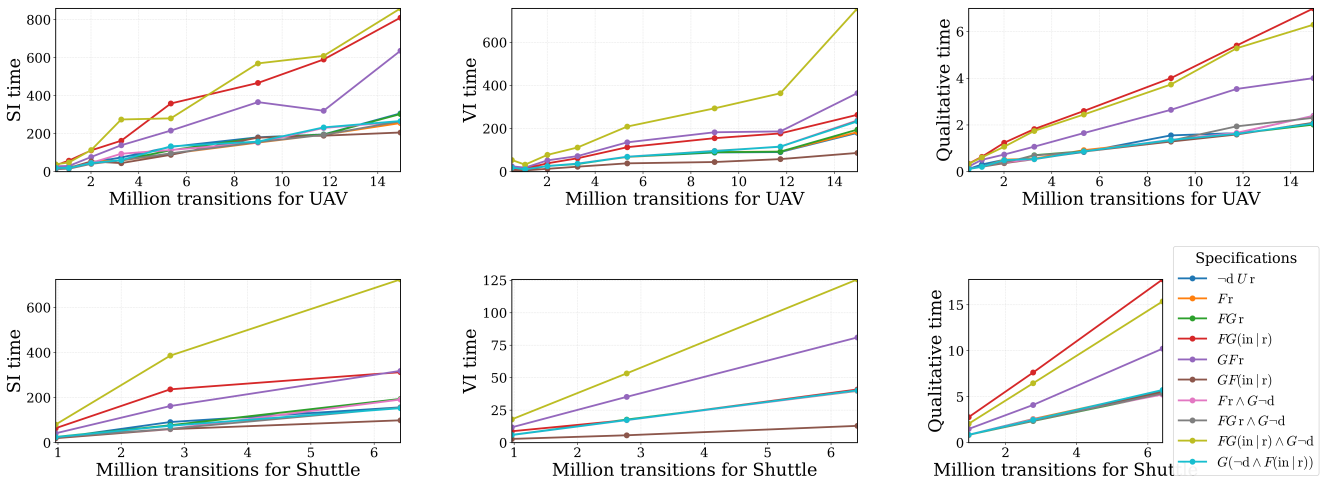



Fig. 3: Performance comparison for the UAV (top row) and Shuttle (bottom row) benchmarks under the selected specifications. The columns correspond to Strategy Improvement (left), Value Iteration (middle), and Qualitative Analysis (right). Times are reported in seconds. The legend is shown only once; the same color-to-specification mapping holds for all plots.

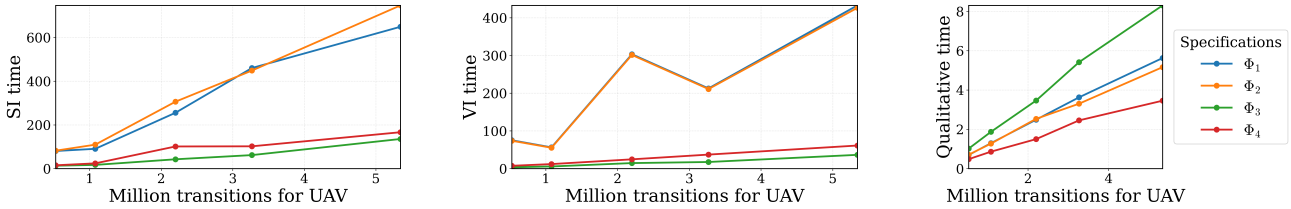



Fig. 4: Performance comparison for the UAV benchmarks under the GR(1) specifications as a function of the number of transitions. Each plot corresponds to Strategy Improvement (left), Value Iteration (middle), and Qualitative Analysis (right). Times are reported in seconds. The legend is shown only once; the same color-to-specification mapping holds for all plots.


[8] T. Brázdil, K. Chatterjee, M. Chmelík, V. Forejt, J. Křetínský, M. Kwiatkowska, D. Parker, and M. Ujma, "Verification of Markov Decision Processes using learning algorithms," in *Proc. of ATVA*, 2014, pp. 98–114.

[9] E. M. Hahn, G. Li, S. Schewe, A. Turrini, and L. Zhang, "Lazy probabilistic model checking without determinisation," in *Concurrency Theory, (CONCUR)*, 2015, pp. 354–367.

[10] T. A. Henzinger and N. Piterman, "Solving games without determinization," in *International Workshop on Computer Science Logic*, 2006, pp. 395–410.

[11] K. Chatterjee, M. Jurdziński, and T. A. Henzinger, "Simple stochastic parity games," in *Proc. of CSL*, 2003, pp. 100–113.

[12] ——, "Quantitative stochastic parity games," in *Proc. of SODA*, 2004, pp. 121–130.

[13] T. A. Henzinger and N. Piterman, "Solving games without determinization," in *Conference on Computer Science Logic*, Szeged, Hungary, 2006, pp. 394–409.

[14] L. de Alfaro and R. Majumdar, "Quantitative solution of omega-regular games," *J. Comput. Syst. Sci.*, pp. 374–397, 2004.

[15] W. Zielonka, "Perfect-information stochastic parity games," in *Proc. of FOSSACS*, 2004, pp. 499–513.

[16] E. M. Hahn, S. Schewe, A. Turrini, and L. Zhang, "A simple algorithm for solving qualitative probabilistic parity games," in *Proc. of CAV*, 2016, pp. 291–311.

[17] ——, "Synthesising strategy improvement and recursive algorithms for solving 2.5 player parity games," in *Proc. of VMCAI*, 2017, pp. 266–287.

[18] K. Chatterjee, M. Jurdzinski, and T. A. Henzinger, "Quantitative stochastic parity games," in *Proc. of SODA*, 2004, pp. 121–130.

[19] C. S. Calude, S. Jain, B. Khoussainov, W. Li, and F. Stephan, "Deciding parity games in quasi-polynomial time," *SIAM J. Comput.*, pp. 17–152, 2022.

[20] K. Lehtinen, P. Parys, S. Schewe, and D. Wojtczak, "A recursive approach to solving parity games in quasipolynomial time," *Log. Methods Comput. Sci.*, 2022.

[21] D. Dell'Erba and S. Schewe, "Smaller progress measures and separating automata for parity games," *Frontiers Comput. Sci.*, 2022.

[22] D. Andersson and P. B. Miltersen, "The complexity of solving stochastic games on graphs," in *Proc. of ISAAC*, 2009, pp. 112–121.

[23] M. Jurdziński, "Deciding the winner in parity games is in UP ∩ co-UP," *Information Processing Letters*, pp. 119–124, 1998.

[24] U. Boker, D. Kuperberg, K. Lehtinen, and M. Skrzypczak, "On the succinctness of alternating parity good-for-games automata," in *Proc. of FSTTCS*, 2020, pp. 41:1–41:13.

[25] S. Haddad and B. Monmege, "Interval iteration algorithm for mdps and imdps," *Theoretical Computer Science*.

[26] A. Nilim and L. E. Ghaoui, "Robust markov decision processes with uncertain transition matrices," 2004.

[27] T. Badings, L. Romao, A. Abate, D. Parker, H. A. Poonawala, M. Stoelinga, and N. Jansen, "Robust control for dynamical systems with non-gaussian noise via formal abstractions," 2023.

[28] A. Duret-Lutz, A. Lewkowicz, A. Fauchille, T. Michaud, É. Renault, and L. Xu, "Spot 2.0 — a framework for LTL and $\omega$-automata manipulation," in *Proc. of ATVA*, 2016, pp. 122–129.

## APPENDIX

### A. Abstraction Algorithms

Here we describe the abstraction algorithms. A schema can be found in Figure 5. We first recall the definition of A discrete-time stochastic control system.

*Definition 3:* A discrete-time stochastic control system (dtSCS) $\Sigma_{ss}$ is a tuple $(\mathcal{S}, U, w, f)$, where

- $\mathcal{S} \subseteq \mathbb{R}^n$ is the continuous state space;
- $U$ is the input space;
- $w$ is a sequence of i.i.d. random variables

$$w := \{w(k) : \Omega \to V_w,\ k \in \mathbb{N}\},$$

  where $\Omega$ is the sample space and $V_w$ is the noise space;
- $f : \mathcal{S} \times U \times V_w \to \mathcal{S}$ is a measurable function describing the state evolution

$$x(k+1) = f(x(k), a(k), w(k)), \tag{2}$$

  where $k \in \mathbb{N}$, $x(k) \in \mathcal{S}$, $a(k) \in U$, and $w(k) \in V_w$.

We denote by $\mathcal{U}_{\mathfrak{a}}$ the set of admissible input sequences

$$\mathcal{U}_{\mathfrak{a}} := \big\{\{a(k) : \Omega \to U,\ k \in \mathbb{N}\}\big\},$$

where, for each $k \in \mathbb{N}$, the input $a(k)$ is independent of $w(t)$ for all $t \geq k$. We assume that the system (2) is unknown but data from sampled trajectories is available.

**IMDP Abstraction.** Depending on the assumptions on $\Sigma_{ss}$ (2), one can construct an IMDP abstraction using sampled data with probabilistic guarantees.

*Assumption 1 (Linear Systems):* Assume $f$ in (2) is linear of the form $f(x, a, w) = Ax + Ba + w$, where $A$ and $B$ are known but the distribution of $w$ is unknown. Moreover, the pair of matrices $[A, B]$ is controllable.

Under Assumption 1, an IMDP abstraction can be constructed using the data-driven procedure shown in Algorithm 4.

*Theorem 1.1 (IMDP abstractions for linear systems):* Under Assumption 1, and with the IMDP computed via Algorithm 4, the number of sampled trajectories can be chosen such that every abstract transition probability is either exactly zero or belongs to an interval whose boundary does not include zero.

Intuitively, Algorithm 4 separates the existence of transitions—determined by the nominal linear dynamics—from the estimation of disturbance probabilities, which reduces to a Bernoulli success-probability estimation problem. With sufficiently many samples, the resulting confidence intervals avoid zero as a boundary point. See [27] for full technical details.

*Assumption 2 (Nonlinear Systems):* Assume $f$ in (2) is nonlinear and unknown, with bound on its Lipschitz constant. The distribution of $w$ is unknown.

Under Assumption 2, an IMDP abstraction can be constructed via the Lipschitz-based over-approximation procedure shown in Algorithm 5.

*Theorem 1.2 (IMDP abstractions for nonlinear systems):* Under Assumption 2, the IMDP constructed using Algorithm 5 may contain probability intervals whose boundary includes zero.

The presence of zero on interval boundaries stems from the conservatism induced by Lipschitz over-approximation (Step 2 of Algorithm 5), which may enlarge reachable sets enough to include transitions that are not observed in sampled data. See [5] for the complete development.

With the abstraction procedures in Algorithms 4 and 5, we obtain an IMDP representation of the underlying dynamical system together with interval-valued transition probabilities. To synthesize a strategy that ensures satisfaction of temporal specifications on such an IMDP, the uncertainty in the transition probabilities must be treated adversarially. This naturally leads to a game-theoretic formulation combined with an automaton encoding of the specification. Therefore, we introduce stochastic parity games and parity automata next.

**Algorithm 4:** Abstraction of Linear Systems as IMDPs

**Input:** Linear $f(x, a, w) = Ax + Ba + w$
**Output:** Abstract IMDP

1 Construct partition $\{S_1, \ldots, S_n\}$ of the state space and select representative points $s_i \in S_i$.
2 **for** *each* $S_i$ *and each* $a \in U$ **do**
3  Use $Ax + Ba$ to test whether $(Ax+Ba) \in S_j$ for some $x \in S_i$.
4  **if** *so* **then**
5  Declare abstract transition $S_i \xrightarrow{a} S_j$.
6 Compute confidence intervals for $\Pr[w \in [\alpha, \beta]]$ from samples.
7 Combine Steps 2–3 to find abstract transition probabilities.

**Algorithm 5:** Abstraction of Nonlinear Systems as IMDPs

**Input:** Nonlinear $f$ with known Lipschitz bound
**Output:** Abstract IMDP

1 Construct partition $\{S_1, \ldots, S_n\}$ of the state space and select representative points $s_i \in S_i$.
2 **for** *each sampled pair* $(x, a)$ **do**
3  Use Lipschitz bound to compute an over-approximation of the forward reachable set.
4 Using sampled trajectories, compute confidence intervals for transition probabilities consistent with the over-approximated reachable sets.
5 Combine reachable-set information and statistical estimates to obtain probability intervals between abstract states $S_i$ and $S_j$.

### B. Proof of Theorem 5.1

*Theorem 5.1:* Given an SMDP $\mathcal{M}$ and automaton $\mathcal{N}$, the qualitative analysis of $\mathcal{M}$ can be done in time polynomial in

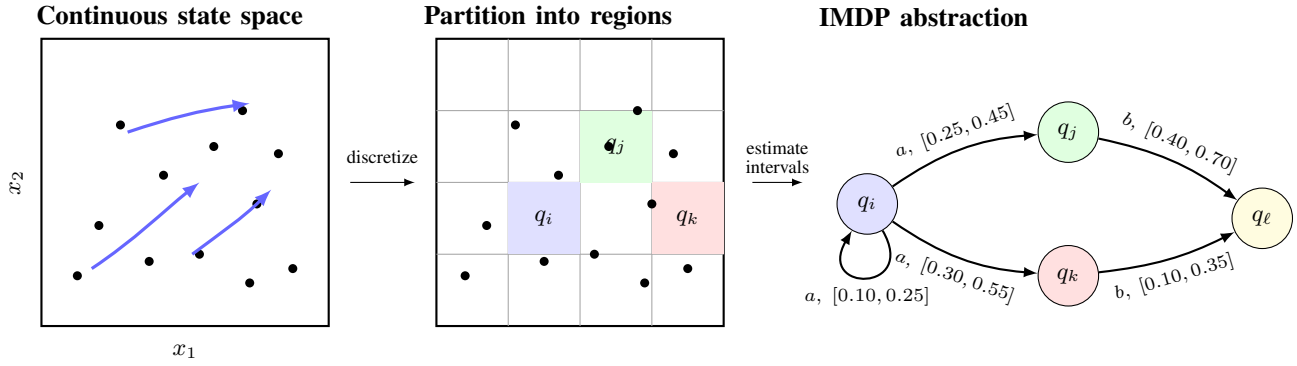



Fig. 5: Construction of an IMDP abstraction from sampled data: the continuous state space is partitioned into finitely many regions, and transition probabilities under each control input are bounded by intervals, yielding a finite model amenable to verification and policy synthesis under uncertainty.

$\mathcal{M}$ and exponential in $\mathcal{N}$ if $\mathcal{N}$ is an NBA. If $\mathcal{N}$ is good for MDPs, this can be done in time polynomial in $\mathcal{M}$ and $\mathcal{N}$.

*Proof:* We first observe that, for the qualitative analysis, only the support of the probability distribution matters, not the probabilities themselves, cf. [18], [16]. As a consequence, the choice of the environment in the game are irrelevant and we can instead work with any distribution from $\Delta(x, a)$. In particular, we can use the same one every time, and as they all have the same support, it does not matter which one we choose. In particular, we can chose arbitrarily and consistently for each state action pair $(x, a)$ for all states $(x, q, a)$ (i.e. independent of the automata state $q$ in the product).

This brings us back to the qualitative analysis of ordinary MDP. For the complexity and hardness for ordinary NBA see [7], [9], for good-for-MDP automata see [6]. ■

### C. Proof of Theorem 5.2

*Proof:* After $n$ steps, the upper bound $V^+$ optimises $\sum_{i=0}^{n} \alpha^i p'_i$, where $p'_i$ for $i < n$ is the chance of reaching the a.s. winning area in precisely $i$ steps, while $p'_n$ is the chance of not having reached the sure losing area after $n$ steps apply. The lower bound $V^-$ optimises $\sum_{i=0}^{n} p_i$, where $p_i$ (including for $i = n$) is the chance of reaching the a.s. winning area in precisely $i$ steps. So, the upper bound converges from above against the discounted reachability value, while the lower bound converges from below to the undiscounted reachability value. As long as not all initial states are in the a.s. winning area or sure losing area, the limit value of the former is lower than the limit value of the latter, which provides termination of the algorithm.

Now, the value obtained is always strictly larger than the discounted reachability value for the given discount factor $\alpha$. Given $\varepsilon$, in order to get suitable $\alpha$, we do the following : pick an $n$ such that $\sum_{i=0}^{n} p_i \geq (1 - 0.5\varepsilon)\mathsf{Pr}_{\mathcal{M}}$ holds for an optimal positional policy to realise $\mathsf{Pr}_{\mathcal{M}}$, and thus also for any policy to optimise $\sum_{i=0}^{n} p_i$, which is what $V^-$ gives after $n$ steps.

Then pick an $\alpha < 1$ s.t. $\alpha^n \geq (1 - 0.5\varepsilon)$. Then $\sum_{i=0}^{n} \alpha^i p_i \geq \sum_{i=0}^{n} \alpha^n p_i \geq \alpha^n \cdot (1 - 0.5\varepsilon)\mathsf{Pr}_{\mathcal{M}} \geq (1 - \varepsilon)\mathsf{Pr}_{\mathcal{M}}$, where the $p_i$ is the chance of reaching the a.s. winning region in $i$ steps. The estimation holds for all policies, and thus also for the supremum. ■

### D. Proof of Theorem 5.3

*Proof:* The proof follows standard fixed-point characterization of stochastic reachability games. Define the operator

$$F(v)(s) = \begin{cases} 1, & s \in W^+, \\ 0, & s \in W^-, \\ \max\limits_{a \in A(s)} \min\limits_{p \in \Delta(s,a)} \sum\limits_{t \in S} p(t)\, v(t), & \text{otherwise,} \end{cases}$$

Its least fixed point is the value vector $V^*$ of the game.

For a fixed positional nature strategy $\theta$, let

$$T_\theta(v)(s) = \begin{cases} 1, & s \in W^+, \\ 0, & s \in W^-, \\ \max\limits_{a \in A(s)} \sum\limits_{t \in S} \theta(s, a, t)\, v(t), & \text{otherwise.} \end{cases}$$

Let $V^\theta$ be the value vector obtained by solving the MDP against $\theta$, which is the least fixed point of $T_\theta$.

Let $(\theta_k, \pi_k, V_k)$ be the nature strategy in the $k$th iteration, $\pi_k$ a best response to $\theta_k$ and $V_k = V^{\theta_k}$. For every state-action pair $(s, a)$,

$$\sum_t \theta_{k+1}(s, a, t)\, V_k(t) = \min_{p \in \Delta(s,a)} \sum_t p(t)\, V_k(t).$$

Hence

$$T_{\theta_{k+1}}(V_k) = F(V_k) \leq T_{\theta_k}(V_k) = V_k.$$

Since $V_{k+1} = V^{\theta_{k+1}}$ is the least fixed point of $T_{\theta_{k+1}}$ and $V_k$ is a post-fixed point, we obtain for all $s$,

$$V_{k+1}(s) \leq V_k(s)$$

Thus $V_i$s are monotonically non-increasing.

If $V_{k+1} = V_k =: V$, then

$$V = T_{\theta_{k+1}}(V) = F(V),$$

So $V$ is a fixed point of $F$, and thus $V = V^*$. Moreover $\pi_{k+1}$ is an optimal policy and $V$ is the optimal value vector.

For each $(s, a)$, the feasible set $\Delta(s, a)$ is a polytope, and the linear minimization is attained at a vertex. Therefore only finitely many positional nature strategies can occur. Since each non-terminal iteration satisfies $V_{k+1} \leq V_k$ and strictly decreases at least one component, no step can repeat before termination. Hence the algorithm terminates after finitely many steps.

■

**Algorithm 6:** Classic Strategy Improvement

**Input:** Regions $W^+$ (a.s. winning), $W^-$ (surely losing)
**Output:** Optimal strategy $\pi^*$ and value vector $V$
$\pi \leftarrow$ arbitrary policy;
$\theta \leftarrow$ arbitrary policy of nature;
$V \leftarrow$ values of states, $V(s) = \mathsf{Pr}^{\pi,\theta}_{\mathcal{M}}(s)$;
**while** $\exists s \notin W^+ \cup W^-$ *s.t.* $\pi(s) \neq \arg\max_{a \in A(s)} \sum_{s' \in S} \theta(s, a, s') V(s')$ **do**
  **foreach** *state* $s \notin W^+ \cup W^-$ **do**
    $\pi(s) \leftarrow \arg\max_{a \in A(s)} \sum_{s' \in S} \theta(s, a, s') V(s')$;
    $\theta \leftarrow$ optimal nature against $\pi$;
    $V \leftarrow$ updated values under $\pi$ and $\theta$;
**return** $\pi, V$;

### *E. Details on Experiments*

This appendix provides a detailed evaluation of the UAV motion-control benchmark introduced in Section VI. We vary the abstraction granularity in the continuous state space, producing SMDPs with 787–2,430 states and $0.52 \times 10^6$–$14.9 \times 10^6$ transitions. All benchmarks are stable IMDPs, ensuring that the quantitative results reported here represent exact optimal probabilities. We present performance trends for qualitative solving, value iteration (VI), and strategy improvement (SI), including convergence time, iteration counts, and value evolution across model sizes. These additional plots complement the summary results in the main text and illustrate the scalability of our techniques on progressively refined abstractions of UAV dynamics.

- **Qualitative solving time.** Figure 6 reports the runtime of the qualitative analysis versus SMDP size (left: states; right: transitions). Transition counts range from $0.52 \times 10^6$ (787 states) to $14.9 \times 10^6$ (2,430 states).
- **VI runtime.** Figure 7 shows VI convergence time versus states and transitions. As the abstraction granularity increases, transitions per state grow and runtime scales accordingly.
- **VI iteration counts.** Figure 8 plots the number of iterations required for VI convergence. Both measures increase with model size, reflecting slower contraction in larger abstractions.
- **VI lower/upper value bounds vs. SI upper bound.** Figure 11 shows the evolution of VI lower and upper bounds on a representative model with 1,600 states. The bounds converge from opposite directions while SI reaches the exact fixed point in only a few iterations.
- **SI runtime.** Figure 9 reports SI convergence time versus states and transitions on the same benchmarks.
- **SI vs. VI runtime ratio.** Figure 10 plots the ratio of SI runtime to VI runtime for each model. Ratios below 1 indicate that SI consistently outperforms VI across all benchmarks.

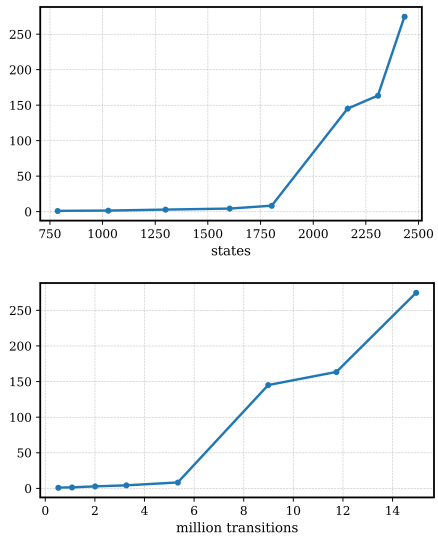



Fig. 6: Qualitative analysis time (seconds) vs. number of transitions (millions) and states

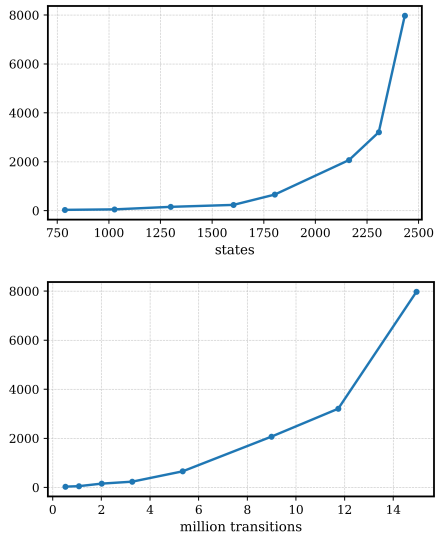



Fig. 7: VI convergence time (seconds) vs. number of transitions (millions) and states

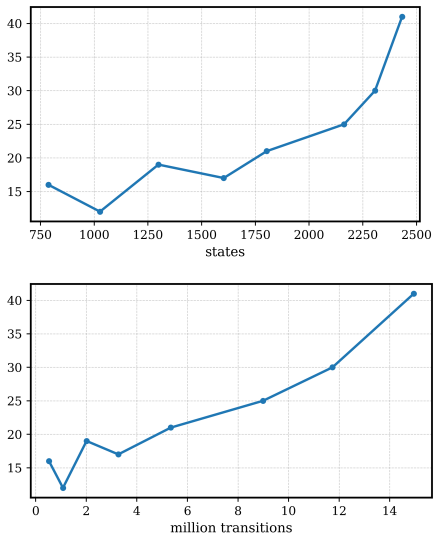



Fig. 8: VI convergence iterations vs. number of transitions and states

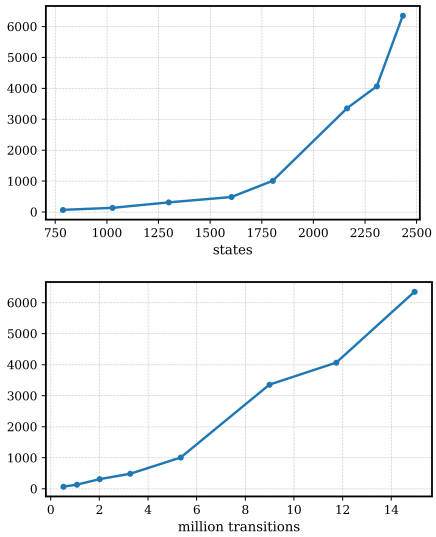



Fig. 9: SI time in seconds vs. number of transitions and states

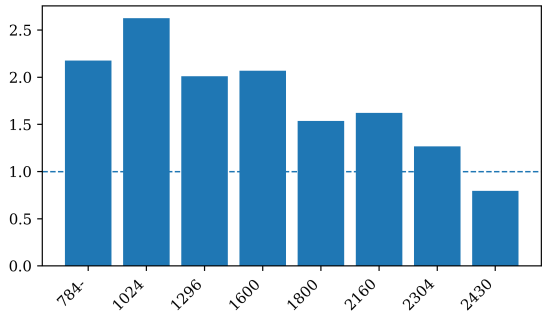



Fig. 10: Ratio of SI time and VI time against number of states

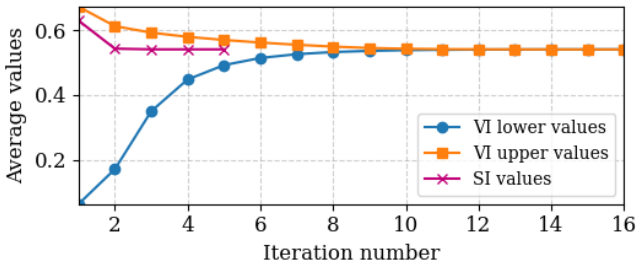



Fig. 11: Average initial-state values for VI lower bounds, VI upper bounds, and SI across iterations for an IMDP with 1600 states under $\mathbf{GF}(reach)$.

### F. Details on GR(1)

In addition to general LTL specifications, we evaluate our framework on the GR(1) fragment, which provides a structured and practically useful class of reactive objectives. A GR(1) specification has the form

$$\Big(\bigwedge_{i=1}^{m} \mathbf{GF}\,\varphi_i\Big) \;\rightarrow\; \Big(\bigwedge_{j=1}^{n} \mathbf{GF}\,\psi_j\Big),$$

where the boolean formulas $\varphi_i$ and $\psi_i$ represent assumptions and guarantees, respectively: if the assumptions hold infinitely often, then the guarantees must hold infinitely often. GR(1) is substantially richer than plain reachability or a single Büchi objective, yet still structured enough to remain tractable. GR(1) specifications naturally induce nondeterministic automata. In the context of IMDPs, GR(1) allows us to express recurring liveness requirements together with safety-sensitive behavior, such as repeated recovery, repeated successful task completion, and avoidance of deadlocks.

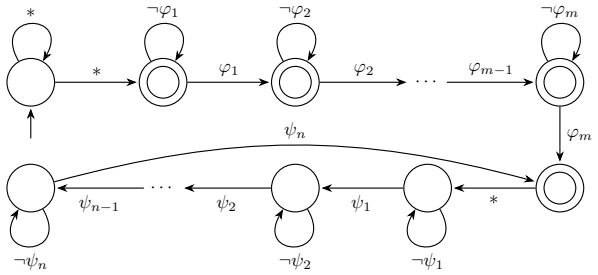



Fig. 12: GR(1) automaton.

Figure 12 shows the automaton schema underlying GR(1) specifications: the upper part corresponds to the assumptions ($\varphi_i$) and the lower part corresponds to the guarantees ($\psi_i$). We then instantiate this pattern with the following four GR(1) specifications over the atomic propositions *init*, *reach*, and *deadlock*:

$$\begin{aligned}
\Phi_1:&\quad \mathbf{GF}(init) \wedge \mathbf{GF}(\neg deadlock) \rightarrow \mathbf{GF}(reach)\\
\Phi_2:&\quad \mathbf{GF}(\neg deadlock) \rightarrow \mathbf{GF}(reach) \wedge \mathbf{GF}(init)\\
\Phi_3:&\quad \mathbf{GF}(\neg deadlock) \rightarrow \mathbf{GF}(reach \wedge \neg init)\\
\Phi_4:&\quad \mathbf{GF}(init \wedge \neg deadlock)\\
&\qquad \rightarrow \mathbf{GF}(reach \wedge \neg deadlock).
\end{aligned}$$

These specifications capture richer recurring mission requirements. $\Phi_1$ requires eventual recurring success under recurring initialization and infinitely often absence of deadlock. The automata that is modeling $\Phi_1$ can be found in Figure 13t. $\Phi_2$ models repeated recovery-and-success behavior, requiring the system to repeatedly revisit both *init* (as recovery) and *reach* (as success). (automata at Figure 14). $\Phi_3$ strengthens this by requiring nontrivial recurring success, namely reaching a progressed goal state distinct from the initial region. (automata at Figure 15). $\Phi_4$ is deadlock-sensitive and requires that whenever the system can repeatedly return to a live initial condition, it must also repeatedly achieve a live success condition. automata at Figure 16).

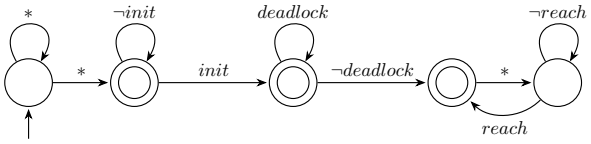



Fig. 13: GR(1)$_1$: $\mathbf{GF}(init) \wedge \mathbf{GF}(\neg d) \rightarrow \mathbf{GF}(r)$.

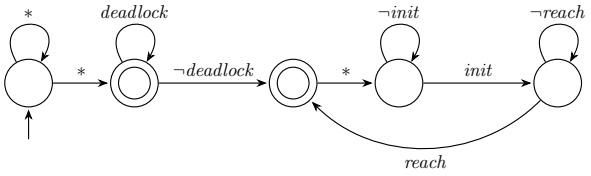



Fig. 14: GR(1)$_2$: $\mathbf{GF}(\neg d) \rightarrow \mathbf{GF}(r) \wedge \mathbf{GF}(init)$.

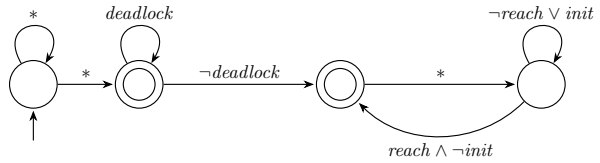



Fig. 15: GR(1)$_3$: $\mathbf{GF}(\neg d) \rightarrow \mathbf{GF}(r \wedge \neg init)$.

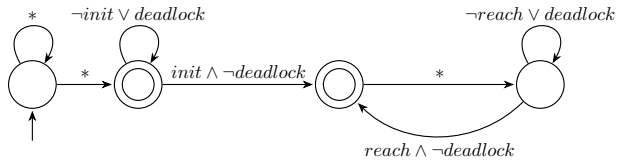



Fig. 16: GR(1)$_4$: $\mathbf{GF}(init \wedge \neg d) \rightarrow \mathbf{GF}(r \wedge \neg d)$.

Fig. 17 reports the runtime for SI, VI, and qualitative analysis for the UAV benchmarks as a function of the number of transitions. These experiments show that our framework handles GR(1)-induced nondeterminism in large IMDPs and can support richer reactive mission requirements than simple reachability-style specifications.

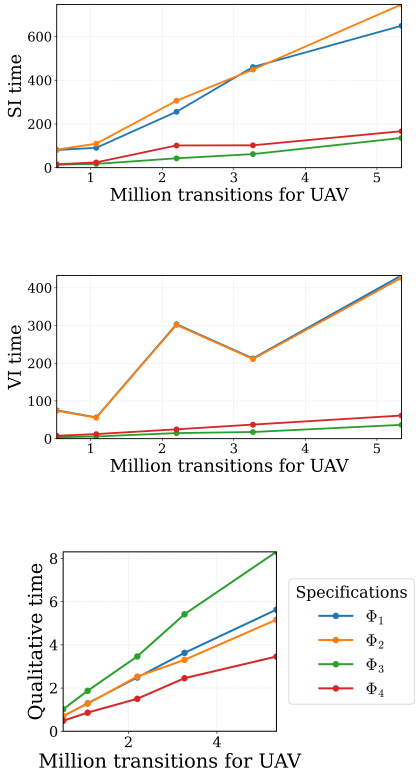



Fig. 17: Performance comparison for the UAV benchmarks under the selected GR(1) specifications. The columns correspond to Strategy Improvement (top), Value Iteration (middle), and Qualitative Analysis (bottom). All times are reported in seconds. The legend is shown only once; the same color-to-specification mapping is used across all subfigures.